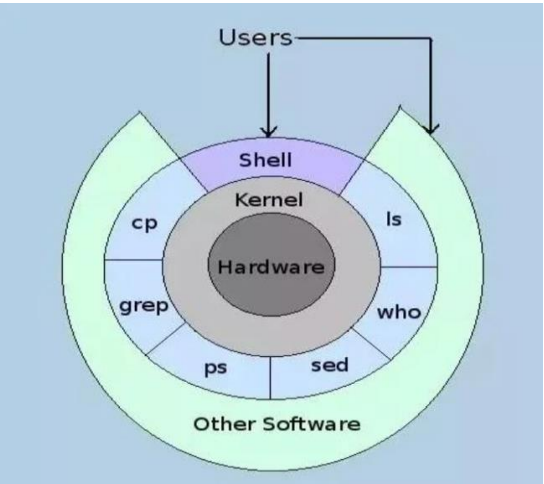
SHELL PROGRAMMING ON LINUX

I. Basic theory

1. Introduction to Shell

A shell is a command-line interface (CLI) that acts as a bridge between the user and the operating system kernel. It enables users to enter commands to perform tasks such as file management, program execution, or automating workflows.



A **shell command** is an instruction given to the shell to perform an operation, such as managing files, executing programs, or handling system processes. Commands are executed in a terminal, and they interact with the operating system through the shell. A shell command generally follows this structure:

```
command [options] [arguments]
```

- **command** – The actual command to execute (e.g., ls, mkdir, echo).
- **options** – Modify the behavior of the command (usually preceded by - or --).
- **arguments** – Specify what the command should act on (e.g., a filename or directory).

Example shell command: ls

```
ls -l /home/user
```

- **ls** → Name of command (using to list directory contents).
- **-l** → The option (long format, showing details like permissions, owner, size, etc.)
- **/home/user** → The argument (the directory to list).

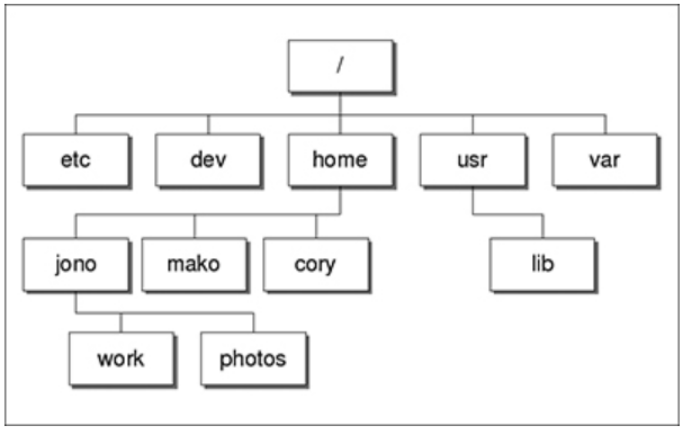
```
File  Edit  View  Search  Terminal  Help
hermione@hermione-VirBox:~$ ls -l /home/hermione
total 72
-rw-rw-r-- 1 hermione hermione     0 Thg 1 22  2023 ABC
-rw-rw-r-- 1 hermione hermione     0 Thg 1  9  2023 ABCD
-rw-rw-r-- 1 hermione hermione     0 Thg 1  9  2023 A_book
-rwxrw-r-- 1 hermione hermione   473 Thg 1  9  2023 add_phonebook
-rw-rw-r-- 1 hermione hermione     0 Thg 1  9  2023 add_phonebook.sh
-rw-rw-r-- 1 hermione hermione     0 Thg 1  9  2023 an_apple
-rw------- 1 hermione hermione   170 Thg 1 19  2023 dead.letter
drwxr-xr-x 5 hermione hermione  4096 Thg 1  4  2023 Desktop
drwxr-xr-x 2 hermione hermione  4096 Thg 8 30  2023 Documents
drwxr-xr-x 2 hermione hermione  4096 Thg 8 30  2023 Downloads
```

Example of executing ls command on terminal

2. Some basic shell commands

2.1 File and directory management shell commands

The **Linux file system** is a structured way of organizing and storing files on a disk. It follows a hierarchical directory structure, starting from the **root directory (/)**. Everything in Linux (files directories, devices, processes) is represented as a file.



Linux filesystem organization

/ (Root Directory) → The top-level directory containing all files and subdirectories.

/home → User directories (/home/user).
/bin → Essential binary programs (e.g., ls, cp, mv).
/etc → Configuration files for the system.
/var → Logs, caches, and variable data.
/tmp → Temporary files (cleared on reboot).
/dev → Device files (e.g., /dev/sda for hard drives).
To locate a file or a directory in Linux file system, we need an absolute path or a relative path.

An **absolute path** is the full path to a file or directory, starting from the root (/).

- Always begins with /.
- Points to the same location, regardless of the current directory.

Example: **/home/jono** is an absolute path, which locate *user directory* ***jono*** in above Linux file system.

A **relative path** is a path relative to the current working directory.

- Does **not** start with /.
- Uses special symbols:
    - . → Current directory
    - .. → Parent directory
    -

Example: If you are in **/home/jono/** and you want to access *subdirectory* ***work*** inside directory jono => **./work**

In Linux, **file access modes** determine who can read, write, or execute a file. These permissions are set for three categories of users and are displayed using the ls -l command.

Each file in Linux has **three types of users** with different access permissions:
Owner (User - u): The person who created the file.
Group (g): Users who belong to the same group as the owner.
Others (o): All other users who are not the owner or in the group.

Each file or directory has **three types of permissions**:

Symbol	Permission	Numeric Value	Description
r	Read	4	Can **view** file contents.
w	Write	2	Can **modify** or **delete** the file.
x	Execute	1	Can **run** the file (if it's a program/script).

Using the ls -l command to display **file permissions**:
Example: ls -l myfile.txt
Example output: -rw-r--r-- 1 user group 1234 Feb 15 12:00 myfile.txt
Owner (rw-) → Can **read** and **write**, but **not execute**.
Group (r--) → Can **only read**.
Others (r--) → Can **only read**.
File permissions can be modified using chmod command
Example: chmod u+x myfile.txt
=>This command adds execute mode access to owner of the file.
Below table summarizes the syntax and usage of basic shell commands for file and directory management.

Command	Syntax	Description	Example usage
cd	cd [directory]	Change directory	cd /home/user /Documents
	cd . .	Move up one level	cd ..
	cd ~	Move to home directory	cd ~
	cd -	Go to the previous directory	cd -
pwd	Pwd	Print current directory path	pwd
mkdir	mkdir [directory]	Create a new directory	mkdir my_folder
	mkdir -p parent/child	Create nested directories	mkdir -p dir1/dir2/dir3
rmdir	rmdir [directory]	Remove an empty directory	rmdir my_folder
cp	cp [source] [destination]	Copy a file	cp file.txt /home/user/backup/
	cp -r [source] [destination]	Copy a directory	cp -r my_folder /home/user/b

			ackup/
mv	mv [source] [destination]	Move a file or directory	mv file.txt /home/user/Documents/
	mv oldname.txt newname.txt	Rename a file	mv old.txt new.txt
Rm	rm [file]	Remove a file	rm file.txt
	rm -r [directory]	Remove a directory and its contents	rm -r my_folder
chmod	chmod [OPTIONS] MODE FILE	**Modify** read (r), write (w), and execute (x) permissions for the **owner (u), group (g), and others (o)**.	chmod u+x script.sh => Execute Permission to Owner chmod u=rw,g=r,o= file.txt => Owner can read & write, group can read, others have no access.
Cat	cat [file]	Display file content	cat file.txt

	cat > [file]	Create a new file	cat > newfile.txt (press Ctrl + D to save)
	cat >> [file]	Append text to a file	echo "New line" >> file.txt
Echo	echo "text"	Print text to the terminal	echo "Hello, World!"
	echo "text" > [file]	Write text to a file	echo "This is a test" > test.txt
	echo $VARIABLE	Print an environment variable	echo $HOME

2.2 Process management shell commands

Basic charateristics of a process

A **process** in Linux is an instance of a running program. Every command or application executed in Linux runs as a process. The Linux kernel manages processes, assigning system resources such as CPU time and memory. Some process characteristics are:

Process ID (PID): A unique identifier assigned to each process.

Parent process ID (PPID): The PID of the parent process that created it.

User ID (UID) & Group ID (GID): Identifies the user and group that owns the process.

Process state: The current state of the process (e.g., running, sleeping, stopped).

CPU & Memory Usage: Amount of CPU time and memory the process consumes.

The ps (**process status**) command is used to display information about active processes running on a Linux system. It helps monitor process details such as process ID (PID), parent process ID (PPID), CPU usage, memory usage, and more.

```
hermione@hermione-VirBox:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
hermione    2461    2427  0 Thg 215 pts/1  00:00:00 bash
hermione    2611    2461  0 00:23 pts/1    00:00:00 ps -f
```

Example of executing ps command in terminal

Redirecting processes in Linux

In Linux, **process redirection** allows you to control the input, output, and error streams of a process. This is useful for saving output to a file, using one command's output as another's input, or discarding unwanted output.

Table below covers types of redirection:

Command	Type	Description	Example usage
>	Output Redirection	Redirects standard output (stdout) to a file (overwrites if exists).	ls > files.txt
>>	Append Output	Redirects output to a file (appends instead of overwriting).	echo "New line" >> log.txt
<	Input Redirection	Redirects input from a file instead of keyboard input.	sort < names.txt
2>	Error Redirection	Redirects standard error (stderr) to a file.	ls non_existent_dir 2> error.log
2>>	Append Error	Appends error messages to a file.	rm nonexistent_file 2>> error.log
&>	Redirect Output & Error	Redirects both stdout and stderr to a file.	command &> output.txt

Interprocess communication (IPC) in Linux

Interprocess Communication (IPC) is a mechanism that allows multiple processes running on the same system (or different systems) to **exchange data and signals** with each other. Since each process in Linux runs in its own memory space, IPC provides ways for processes to share information and synchronize their execution.

A **pipe (|)** in Linux is an **interprocess communication (IPC) mechanism** that allows the output of one process to be used as input for another process. Pipe are **unidirectional** (data flows in one direction).

Example: Find all file have extension ".txt" in current directory using grep, ls and pipe

Grep command

The **grep** command in Linux is used for searching **text or patterns** within files.

Syntax of grep:

grep [OPTIONS] PATTERN [FILE...]

- PATTERN: The search pattern (string or regex).
- FILE: The file(s) to search within.
- OPTIONS: Modify search behavior (e.g., case-insensitive, recursive search).

Summary table of using grep

Option	Description	Example
grep "pattern" file	Search for "pattern" in file	grep "error" logfile.txt
-i	Case-insensitive search	grep -i "error" logfile.txt
-n	Show line numbers	grep -n "error" logfile.txt

-v	Show lines **not** matching pattern	grep -v "error" logfile.txt
-w	Match whole word only	grep -w "error" logfile.txt
-c	Count occurrences	grep -c "error" logfile.txt
-r	Search recursively in a directory	grep -r "error" /var/logs/

Example: Find all file have extension ".txt" in current directory
ls | grep ".txt"
ls command lists all files in current directory and passes the output to grep using pipe (|).Then, grep command filters only .txt files in current directory.

A **signal** is a software-generated **notification** sent from one process to another or from the OS to a process.

Signal	Number	Description
SIGKILL	9	Immediately terminates a process (cannot be ignored).
SIGTERM	15	Requests a process to terminate (default kill signal).
SIGSTOP	19	Pauses (stops) a process.

SIGCONT	18	Resumes a stopped process.
SIGINT	2	

Example: Sending signals using kill command

Find a process ID (PID): ps -e | grep firefox

Kill a process (sending SIGKILL signal to process): kill -9 <PID>

⇨ This forcefully stops the process

3. Introduction to shell script

Shell script is a collection of commands written in a text file to perform a specific task. When executed, the shell reads and executes each line of the script sequentially.

Popular shells:

- bash (Bourne Again Shell): The default shell on most Linux systems.
- sh (Bourne Shell): Simple and lightweight.
- zsh (Z Shell): Advanced features.
- csh (C Shell): Ideal for mathematical scripting.

3.1 Creating and Running Shell Scripts

Step 1. Create a script file

- Using a text editor like vim, nano or gedit.
- Example: vim scriptname.sh
- A script file with the name myscript.sh is created.

Step 2. Write commands in the script

Vim has three different modes:

Mode	Usage	How to enter

Normal mode (default mode)	Used for navigation and commands) - Using arrow keys to move left, right, down, up - Copy + yy: copy the current line + yw: copy the current word - Paste + p: paste after cursor +P: paste before cursor - Delete: + dd: delete current line + dw: delete current word - Undo and redo: + u: undo last change +Ctrl plus r: redo last undone change	Press Esc
Insert mode	Type/edit text, can use arrow keys to navigate	Press i or a
Command mode	Run Vim commands - :w -> save the file - :q -> quit Vim - :wq or zz save and quit - :q! ->quit without saving - :!command -> run a shell command in Vim (Ex: :!ls)	Press : from normal mode

- The first line should start with #!/bin/bash to specify the shell interpreter.
- Write commands to solve the problem.
- Save and exit the editor:
    - nano: Press CTRL + X, then Y, and Enter.
    - vim: Press ESC, type :wq, and press Enter.

Step 3. Grant execute permissions

- Using command: chmod +x [scriptname].sh

Step 4. Run the script

- Using command: ./[scriptname].sh

3.2. Basic Structure of Shell scripts

3.2.1 Variables in Shell

You don't usually declare variables in the shell before using them. Instead, you create them by simply using them (for example, when you assign an initial value to them). You can access the contents of a variable by preceding its name with a $. A string must be delimited by quote marks if it contains spaces. In addition, there can't be any spaces on either side of the equals sign.

You can assign user input to a variable by using the read command and use echo command to display value of a variable.
Example:

```
$ salutation=Hello
$ echo $salutation
Hello
$ salutation="Yes Dear"
$ echo $salutation
Yes Dear
$ salutation=7+5
$ echo $salutation
7+5
```

```
$ read salutation
Wie geht's?
$ echo $salutation
Wie geht's?
```

Environment variables and Parameter variables

When a shell script starts, some variables are initialized from values in the environment. These are normally in all uppercase form to distinguish them from user-defined (shell) variables in scripts, which are conventionally lowercase.

Environment Variable	Description
$HOME	User's home directory.
$USER	Current username.
$PWD	Current working directory
$#	The number of parameters passed
$0	The name of the shell script

If your script is invoked with parameters, some additional variables are created. If no parameters are passed, the environment variable $# still exists but has a value of 0.
The parameter variables are listed in the following table:

Parameter Variable	Description
$1, $2,….	The parameters given to the script
$@	A list of all parameters

Example: Manipulating parameter and enviroment variables

Shell script	Result

`#!/bin/sh`
 	
`salutation="Hello"`
`echo $salutation`
`echo "The program $0 is now running"`
`echo "The second parameter was $2"`
`echo "The first parameter was $1"`
`echo "The parameter list was $*"`
`echo "The user's home directory is $HOM`
`echo "Please enter a new greeting"`
`read salutation`
 	
`echo $salutation`
`echo "The script is now complete"`
`exit 0`
	`$ ./try_var foo bar baz`
	`Hello`
	`The program ./try_var is now running`
	`The second parameter was bar`
	`The first parameter was foo`
	`The parameter list was foo bar baz`
	`The user's home directory is /home/rick`
	`Please enter a new greeting`
	`Sire`
	`Sire`
	`The script is now complete`
	`$`

3.2.2 Conditions

Fundamental to all programming languages is the ability to test conditions and perform different actions based on those decisions. The condition types that you can use fall into three types: string comparision, arithmetic comparison and file conditionals. The following tables describes these condition types:

String Comparison	Result
`string1 = string2`	True if the strings are equal
`string1 != string2`	True if the strings are not equal
`-n string`	True if the string is not null
`-z string`	True if the string is null (an empty string)
Arithmetic Comparison	**Result**
`expression1 -eq expression2`	True if the expressions are equal
`expression1 -ne expression2`	True if the expressions are not equal
`expression1 -gt expression2`	True if `expression1` is greater than `expression2`
`expression1 -ge expression2`	True if `expression1` is greater than or equal to `expression2`
`expression1 -lt expression2`	True if `expression1` is less than `expression2`
`expression1 -le expression2`	True if `expression1` is less than or equal to `expression2`
`! expression`	True if the expression is false, and vice versa
File Conditional	**Result**
`-d file`	True if the file is a directory
`-e file`	True if the file exists. Note that historically the `-e` option has not been portable, so `-f` is usually used.
`-f file`	True if the file is a regular file

In practice, most scripts make extensive use of the [or test command, the shell's Boolean check. On some systems, the [and test commands are synonymous, except that when the [command is used, a trailing] is also used for readability. Note that you must put spaces between the [braces and the condition being checked.

Example: Checking whether a file exists using [ ] and test

if test -f myfile.txt then ..... fi	if [-f myfile.txt] then ..... fi

3.3.3. Arithmetic Expansion

Table below covers some common expression evaluations:

Expression Evaluation	Description
`expr1 | expr2`	`expr1` if `expr1` is nonzero, otherwise `expr2`
`expr1 & expr2`	Zero if either expression is zero, otherwise `expr1`
`expr1 = expr2`	Equal
`expr1 > expr2`	Greater than
`expr1 >= expr2`	Greater than or equal to
`expr1 < expr2`	Less than
`expr1 <= expr2`	Less than or equal to
`expr1 != expr2`	Not equal
`expr1 + expr2`	Addition
`expr1 - expr2`	Subtraction
`expr1 * expr2`	Multiplication
`expr1 / expr2`	Integer division
`expr1 % expr2`	Integer modulo

To evaluate an arithmetic expression, you can use expr command or enclosing the expression by $((..))

Example:

```
X=0
X=$(( $x + 1))
```

```
X=0
X=$(expr $x +1)
```

3.3.4 Command execution

When you're writing scripts, you often need to capture the result of a command's execution for use in the shell script; that is, you want to execute a command and put the output of the command into a variable. You can do this by using the $(command). The result of the $(command) is simply the output from the command. Note that this isn't the return status of the command but the string output.

Example:

```
X=0
```

X=$(expr $x +1) ⇨ expr command is executed and stored result to variable x

Note: The $((…)) are used for arithmetic substitution. The $(…) is used for executing commands and grabbing the output.

3.3.5 Control structures

The shell has a set of control structures, which are very similar to other programming languages.

If

The if statement is very simple: It tests the result of a command and then conditionally executes a group of statements:

Syntax	Example

if condition
then
Statements
else
Statements
fi
	`#!/bin/sh`
	
	`echo "Is it morning? Please answer yes or no"`
	`read timeofday`
	
	`if [ "$timeofday" = "yes" ]`
	`echo "Good morning"`
	`else`
	`echo "Good afternoon"`
	`fi`
	
	`exit 0`

For

Use the for construct to loop through a range of values, which can be any set of strings.

Syntax	Example

for variable **in** values
do
statements
done
	`#!/bin/sh`
	
	`for foo in bar fud 43`
	`do`
	`echo $foo`
	`done`
	`exit 0`
	The results in the following output:
	bar fud 43

	for var in {1..5} do echo $var

	done

While

When you need to repeat a sequence of commands, but don't know in advance how many times they should execute, you will normally use a while loop.

Syntax	Example

while condition **do**
  statements
done
	`#!/bin/sh`
	
	`echo "Enter password"`
	`read trythis`
	
	`while [ "$trythis" != "secret" ]; do`
	`  echo "Sorry, try again"`
	`  read trythis`
	`done`
	`exit 0`

	count=1
	while [$count -le 5]; do
	  echo $count
	  count=$((count+1))
	done

Until

This is very similar to the while loop, but with the condition test reversed. In other words, the loop continues until the condition becomes true, not while the condition is true.

Syntax	Example

until condition
do
  Statements
done
	num=1
	until [$num -ge 5]; do
	  echo $num
	  num=$((num+1))

	done

Case

Case construct enables you to match the contents of a variable against patterns and then allows execution of different statements, depending on which pattern was matched

Syntax	Example

case variables **in**
pattern [| pattern] ….)
statements ;;
pattern [| pattern] ….)
statements ;;
…
esac
	`#!/bin/sh`
	
	`echo "Is it morning? Please answer yes or no"`
	`read timeofday`
	
	`case "$timeofday" in`
	`  yes | y | Yes | YES )`
	`  echo "Good Morning"`
	`  echo "Up bright and early this morning"`
	`  ;;`
	`  [nN]*)`
	`  echo "Good Afternoon"`
	`  ;;`
	`  *)`
	`  echo "Sorry, answer not recognized"`
	`  echo "Please answer yes or no"`
	`  exit 1`
	`  ;;`
	`esac`
	
	`exit 0`

You must be careful to put the most explicit matches first and the most general match last. This is important because case executes the first match it finds, not the best match.

3.3.6 Function

You can define functions in the shell; and if you write shell scripts of any size, you'll want to use them to structure your code. To define a shell function, simply write its name followed by empty parentheses and enclose the statements in braces.

When a function is invoked, the positional parameters to the script, $*, $@, $#, $1, $2, and so on, are replaced by the parameters to the function. That's how you read the parameters passed to the function. When the function finishes, they are restored to their previous values.

Syntax	Example

function_name () {
    statements
}

```
sum() {
   echo $(( $1 + $2 ))
}
result=$(sum 5 3)
echo "Result: $result"
```

```
#!/bin/sh

yes_or_no() {
  echo "Is your name $* ?"
  while true
  do
    echo -n "Enter yes or no: "
    read x
    case "$x" in
      y | yes ) return 0;;
      n | no )  return 1;;
      * )       echo "Answer yes or no"
    esac
  done
}
echo "Original parameters are $*"

if yes_or_no "$1"
then
  echo "Hi $1, nice name"
else
  echo "Never mind"
fi
exit 0
```

Typical output from this script might be as follows:

```
$ ./my_name Rick Neil
Original parameters are Rick Neil
Is your name Rick ?
Enter yes or no: yes
Hi Rick, nice name
$
```

4. Debugging and Optimizing Shell scripts

Debugging

```
bash -x script.sh
```

Optimizing

- Use functions to avoid repetition.
- Use built-in shell commands for efficiency.

5. Look up the syntax of command

On Linux, you can use the following commands to look up the syntax and usage of a command or program:

- "man" command

```
man <command>
```

Example: man ls

- If you're unsure which category a command belongs to, search for related topics:

```
man -k <keyword>
```

- "info" command (Some commands have an info page)

```
info <command>
```

- --help option (quick usage summary)

```
<command> --help
```

6. Additional Resources

- "The Linux Programming Interface" - Michael Kerrisk
- "The Linux Command Line" - William Shotts

II. Exercises

File and directory management shell command

Exercise 1: Execute following requests using shell command

1. Move to your user directory

2. Inside your user directory, create two subdirectories **Slides**, **Labs** and a file name **Solution.txt**
3. Go to **Labs** directory
4. Inside **Labs** directory, create a subdirectory named **Lab1** and a file named **Ex1.txt**
5. Copy file **Ex1.txt** from labs directory to **Lab1** directory.
6. Remove file **Ex1.txt** inside **Labs** directory
7. Rename of file **Ex1.txt** to **Exercise1.txt**
8. Add write mode access to file **Exercise1.txt** for other users
9. Remove directory **Slides**

Exercise 2: Continue exercise 1, go to **Lab1** directory and try this sequence of commands and record results of these commands.

1. cd
2. pwd
3. ls –al
4. cd .
5. pwd
6. cd ..
7. pwd
8. ls -al
9. cd ..
10. pwd
11. ls -al

Exercise 3: Create file **Hello.txt** inside Lab1 directory. Then using echo command and redirecting to write below contents to file Hello.txt.

Hello everyone!

This is Advanced Programming Techniques course.

There are many exercises for you in this course.

Let's practice what we learn in this course with exercises.

Are you ready?

Process management shell command

Exercise 4: Use the I/O redirection and pipe mechanisms and Unix commands to perform the following tasks:

1. Count number of word **course** appeared in file Hello.txt (you created in exercise 3) and save results to file Ex4_results.txt.
2. Print 3rd line of the file Hello.txt, then count the number of words in this line and save results to the end of file Ex4_results.txt.
   (Hint: using sed, wc and pipe mechanism)
3. Store the names and attributels s of the directories and files inside your user directory.
4. Count total the number of regular files and directories inside your user directory and save it to the end of file Ex4_results.txt.
5. Count total the number of directories inside your user directory and save it to the end of file Ex4_results.txt.
6. Count the total number of processes currently in the system and save results to the end of file Ex4_results.txt

Shell script

Exercise 5: Write a set of commands in a shell script file named **greeting.sh** that does the following:

a) Contains a comment with your name, the name of this script, and the purpose of this script.

b) Displays the string "Hello + username"

c) Displays the name and version of this operating system.

d) Displays the group ID (GID) of the current user.

e) Displays the current date and time.

h) Displays a list of all the files in the current user's directory.
i) Displays the values of the TERM, PATH, and HOME variables,
j) Displays "Goodbye + current time" to the user.

Exercise 6: Write a shell script named **test.sh** (with with any number of parameters and parameter list) that displays the following information:
- Program name
- Number of command line parameters
- First command line parameter entered by the user
- All command line parameters entered by the user

Exercise 7: Write a shell script to convert a decimal integer to hexadecimal. The shell script takes a command line parameter which is a decimal integer to convert. *Hint:* using **bc** command to convert from decimal to hexadecimal

Exercise 8: Program a shell script to print the Fibonacci sequence up to the nth number. The Fibonacci sequence has the following characteristics:
- F(0) = 0
- F(1) = 1
- F(n) = F(n-1) + F(n-2) with n > 1

● Requirements:
- Enter the value n from the user.
- Print the Fibonacci sequence from F(0) to F(n).